

**PortfolioCheck: Design, Development and Evaluation
of a Web Application for the Automated Verification
of Formal Coursework Submission Requirements**

Course: Project: Software Engineering (DLMCSPSE01)

Portfolio Part 3 — Finalisation Phase

Degree Programme: Master of Science (M.Sc.)

Name of Author: Siva Kondaru

Matriculation Number: 10243569

Name of Tutor: Holger Klus

Date: October 2026

Abstract

This document is the complete project documentation of *PortfolioCheck*, a web application that verifies a coursework portfolio submission against the formal requirements of its assignment brief before the student hands it in. It consolidates the conception and development phases, revised in response to the tutorial feedback, and adds the implementation, testing and deployment results of the finalisation phase. A student uploads the zip folder they intend to submit; the application evaluates it against seven rules derived from the brief and returns a report in which every problem is paired with the exact correction required and with the provision of the brief it derives from. The most distinctive rule compares the submitted code archive with the public GitHub repository file by file, using the content hashes that git itself uses. The project was managed with Kanban, with the board moved into GitHub Projects after the Phase 2 feedback, the architecture is documented in a UML component and a UML deployment view, and the application is verified by 73 automated tests and thirteen manual test cases. All thirteen functional requirements are implemented. The document closes with the remaining technical debt and the lessons learned.

Keywords: software engineering, requirements engineering, software architecture, UML, Kanban, software testing, technical debt

Contents

Abstract	II
List of Abbreviations	IV
1 Introduction	1
1.1 Problem and Objective	1
1.2 Revisions after the Phase 2 Feedback	1
1.3 Theoretical Background	1
2 Project Management	3
2.1 Project Profile	3
2.2 Work Packages and Timeline	3
2.3 Kanban in GitHub Projects	4
2.4 Risks	5
3 Requirements and Architecture	6
3.1 Requirements	6
3.2 Glossary	7
3.3 Architecture Views	7
3.4 Technology Stack	9
4 Implementation and Evaluation	10
4.1 Implementation	10
4.2 Testing	11
4.3 Results	13
4.4 Deployment and Installation	15
5 Reflection	15
5.1 Technical Debt	15
5.2 Lessons Learned	16
5.3 Conclusion	16
References	18
Appendix A: Working Results	20

List of Figures

1	Kanban board in GitHub Projects with the work packages WP1–WP11	5
2	Component view (UML component diagram)	8
3	Deployment view (UML deployment notation)	9
4	Report for sample M05 checked as phase 3	14
A.1	Start page with upload form and rule catalogue	20
A.2	M01, phase 1: compliant submission — <i>Ready to submit</i>	21
A.3	M02, phase 1: underscore in the main directory name — R01 fails	21

A.4	M03, phase 1: folder 02_requirements — R02 fails and names it	22
A.5	M04, phase 1: six-digit matriculation number — R01 warns, <i>Almost there</i>	22
A.6	M06, phase 3: code archive named code.zip — R03 fails	23
A.7	M07, phase 3: links file without application URL — R04 warns	23
A.8	M08: archive containing a ../ path — rejected before extraction	24
A.9	M09: text file renamed to .zip — rejected as invalid	24
A.10	Rule catalogue at /rules: all seven rules with their phases and sources	25
A.11	Health endpoint at /health, used by the cloud host to monitor the service	25
A.12	The application deployed on Render	25
A.13	Automated test run: all 73 tests pass	26

List of Tables

1	Phase 2 feedback and how it was addressed	2
2	Work packages with dates, deliverables and requirements	4
3	Requirements with priority and verifying evidence	6
4	Components and the source modules that implement them	8
5	Manual test cases with input, expected behaviour and result	12
6	Open technical debt	15

List of Abbreviations

Abbreviation	Definition
API	Application Programming Interface
FR	Functional Requirement
HTTP	Hypertext Transfer Protocol
ISO	International Organization for Standardization
NFR	Non-Functional Requirement
PDF	Portable Document Format
REST	Representational State Transfer
TD	Technical Debt
UML	Unified Modeling Language
WIP	Work in Progress
WP	Work Package

1 Introduction

1.1 Problem and Objective

Every portfolio submission in a distance-learning degree programme is governed by purely formal provisions. The brief of this course alone prescribes a naming pattern for the main directory, a second pattern for the code archive and a third for each phase document; four subdirectories whose names must match exactly; permitted file formats; a size limit of 15 MB; a links file with the repository address, the application address and any login data; and the requirement that the public repository and the submitted code archive be identical. These provisions exist so that examination offices can process large numbers of submissions consistently — the same pressure of scale that drives automation elsewhere in educational assessment ([Paiva et al., 2022](#)).

The provisions are deterministic, verifiable without human judgement and numerous, and they are checked at the worst possible moment: in the final hours before a deadline, when the attention available for mechanical detail is at its lowest ([Hermans, 2021](#)). Research on automated assessment concentrates on the *content* of student work, above all the correctness of program code ([Messer et al., 2024](#); [Combéfis, 2022](#)); the formal envelope of a submission has received almost no attention. Students therefore check it by hand, without any confirmation of success, and discover errors only when they can no longer be corrected.

The objective of the project was a web application that accepts the archive a student intends to submit, evaluates it against the formal requirements, and returns a report stating for each requirement whether it is met and, if not, exactly what must be changed. That last element is decisive: feedback that names the correction is what gives automated assessment its practical value, whereas a bare verdict merely restates the problem ([Combéfis, 2022](#)).

The document follows the course of the project. Section 1.3 summarises the theory the project builds on. Chapter 2 describes how the project was organised: its profile, the work packages and their dates, the Kanban workflow and the risks. Chapter 3 states what the system must do and how it is built: the requirements, the glossary, two UML architecture views and the technology stack. Chapter 4 reports what was built and shows that it works: the implementation, the manual and automated tests, the results and the deployment. Chapter 5 closes with the technical debt, the lessons learned and a conclusion. This order mirrors the life cycle of a software product, from planning to evaluation ([Sommerville, 2020](#)).

1.2 Revisions after the Phase 2 Feedback

This document revises the Phase 2 document and completes it for the final submission. Table 1 shows how each point of the Phase 2 feedback was addressed, so that the development of the work remains traceable across the portfolio phases ([Bass et al., 2021](#)).

1.3 Theoretical Background

This section summarises the concepts from software engineering and related fields on which the design decisions of the following chapters rest. Each concept is introduced only as far as the project uses it.

Automated assessment and feedback. Automated assessment tools evaluate student work against a specification without a human grader, which makes the evaluation fast, repeatable and available at any time ([Paiva et al., 2022](#)). Their pedagogical value, however, depends less on the verdict than on

Table 1: Phase 2 feedback and how it was addressed

No.	Feedback	How it was addressed
1	Include specific dates in the project plan	Every work package and milestone has calendar dates (Table 2).
2	Kanban figure and work-package table inconsistent	The board holds exactly the eleven work packages of Table 2, in the same states.
3	Explain the day-to-day use of the board and the tool	Section 2.3 describes the workflow in GitHub Projects.
4	Screenshots of the real board instead of a drawing	Figure 1.
5	Improve the presentation of the architecture	Two UML views, each with a stated purpose, a legend and a mapping to the source code (Section 3.3).
6	Test cases with specific input values and expected behaviour	Thirteen manual test cases with reproducible input files (Table 5).
7	Technical debt is not a list of unimplemented requirements	Rewritten as implementation shortcuts (Table 6).

the feedback attached to it: reviews of grading tools consistently find that a bare pass or fail helps a student far less than a message that locates the problem and states how to correct it (Messer et al., 2024). Most tools of this kind judge program behaviour through test cases or static analysis; checking the formal envelope of a submission is a neighbouring but distinct problem, because the rules are fixed by an examination regulation rather than by a programming exercise (Comb  fis, 2022). PortfolioCheck adopts the feedback principle of this literature and applies it to that envelope.

Requirements and quality models. Requirements engineering separates what a system must do, the functional requirements, from the qualities with which it must do it, the non-functional requirements. A requirement is only useful if it is verifiable, that is, if a test or review can show whether it is met (Sommerville, 2020). Non-functional requirements are easier to state completely when they follow a quality model; ISO/IEC 25010 structures product quality into characteristics such as security, performance efficiency, usability, maintainability, reliability and portability (ISO/IEC, 2023). Agile methods add prioritisation: the MoSCoW scheme ranks requirements as *Must*, *Should*, *Could* or *Won't*, so that the scope can shrink under time pressure without losing the core (Ambler and Lines, 2022).

Software architecture. A software architecture describes the components of a system, their responsibilities and the dependencies between them, and it is judged by the quality attributes it enables (Bass et al., 2021). Layered architectures that place the domain model at the centre and let it depend on nothing keep the business rules testable without infrastructure; dependencies point inwards, towards the more stable code (Percival and Gregory, 2020). Every architectural style is a trade-off: a monolith deployed as one unit is simpler to build, test and operate than distributed services, which often makes it the better choice for a small system built by a single team (Richards and Ford, 2020).

Secure handling of untrusted input. Any data that crosses the system boundary must be treated as hostile until it has been validated; injection and insecure design rank among the most common causes of web application vulnerabilities (OWASP, 2021). Archives pose two specific threats. A *path traversal* entry such as `../../etc/passwd` writes outside the extraction folder when the archive is unpacked, and a *decompression bomb* expands a small upload into gigabytes that exhaust memory or disk. Secure development frameworks therefore recommend secure coding practices that validate all input before it is processed, which for archives means inspecting them against explicit limits before

anything is extracted ([NIST, 2022](#)).

Content addressing in version control. Git stores every file as a *blob* identified by the SHA-1 hash of a short header and the file's content. Two files with the same hash have the same content, whatever their name or timestamp, which is what lets git detect changes without comparing files byte by byte ([Chacon and Straub, 2024](#)). Because the GitHub REST API returns these hashes for every file of a repository tree, the content of a repository can be compared with local files without downloading them ([GitHub, 2025a](#)).

Testing. Automated tests are commonly arranged as a pyramid: many fast unit tests at the base, fewer integration tests above them and only a few end-to-end tests at the top, because the cost and fragility of a test grow with the amount of the system it exercises ([Winters et al., 2020](#)). A good unit test protects against regressions, resists refactoring and runs fast; external services are replaced by test doubles so that the suite does not depend on the network ([Khorikov, 2020](#)).

Process and technical debt. Kanban manages work as a flow: work items move across a visible board, work in progress is limited, and the process is improved continuously, without the fixed roles and events of Scrum ([Kim et al., 2021](#); [Schwaber and Sutherland, 2020](#)). Technical debt describes the shortcuts taken in the implementation that make later change more expensive; like financial debt, it accrues interest until it is repaid ([Avgeriou et al., 2023](#)). Managing it means recording each item and prioritising repayment by its effect on further development ([Lenarduzzi et al., 2021](#)).

2 Project Management

2.1 Project Profile

The application judges *form and never content*, and it *stores nothing*: an uploaded archive is processed and deleted within a single request. Grading of content, plagiarism detection, integration with the examination platform and user accounts are outside the scope; omitting accounts in particular removes a whole class of security and data-protection obligations, since a smaller attack surface leaves fewer weaknesses to defend ([OWASP, 2021](#)).

Four measurable objectives were set at the start and used to judge the result. The application must check every formal provision of the brief that can be checked mechanically; every failure must come with the exact correction; a typical submission must be checked within three seconds; and the application must be publicly reachable without any installation or login. Stating objectives in a form that can later be verified is what allows a project to be evaluated against its own intentions rather than against impressions ([Sommerville, 2020](#)). Section 4.3 returns to each of them.

The target group is students in distance-learning programmes who prepare portfolio submissions independently, above all those submitting for the first time and those working close to a deadline. The examination office benefits indirectly but is not a user and therefore not a source of requirements; separating beneficiaries from users keeps the scope from growing unnecessarily ([Sommerville, 2020](#)).

The project was carried out by one student occupying every role. The roles were kept distinct nonetheless, because the activities differ in kind ([Sommerville, 2020](#)): the *product owner* prioritised requirements, the *architect* decided the structure and the technologies, the *developer* implemented the components, and the *tester* specified the test cases and prepared the test data. The tutor provided feedback at the end of each phase.

2.2 Work Packages and Timeline

The work was divided into eleven work packages across the three portfolio phases. Table 2 gives each work package with its dates, its deliverable and the requirements it delivers, so that the plan can be read directly against the requirements in Table 3 ([Ambler and Lines, 2022](#)).

Table 2: Work packages with dates, deliverables and requirements

WP	Work package	Deliverable	Requirements	Dates (2026)
Phase 1 — Conception				
WP1	Analysis and profile	Project profile, target group, risks	—	10.08.–16.08.
WP2	Requirements	Requirements, glossary, test strategy	all	13.08.–20.08.
WP3	System design	Architecture views, technology decisions	NFR7, NFR9	17.08.–23.08.
<i>Milestone M1: Portfolio Part 1 submitted on 23.08.2026</i>				
Phase 2 — Development				
WP4	Domain model and rule engine	Domain types, rule registry, evaluation	FR7–FR9, NFR7, NFR8	24.08.–06.09.
WP5	Archive extraction	Extraction with safety limits	FR1, NFR1–NFR3	31.08.–06.09.
WP6	Rule set	Rules R01–R06	FR2–FR6, FR13	31.08.–13.09.
WP7	Web interface	Upload, report, catalogue, skeleton	FR1, FR10, FR11, NFR6	07.09.–17.09.
WP8	Test suite	Automated tests	NFR1, NFR2, NFR5, NFR8	24.08.–17.09.
<i>Milestone M2: Portfolio Part 2 submitted on 18.09.2026</i>				
Phase 3 — Finalisation				
WP9	Repository comparison	RepositoryClient and rule R07	FR12	21.09.–27.09.
WP10	Deployment	Container, cloud deployment	NFR4, NFR9	28.09.–04.10.
WP11	Finalisation	Test data, test cases, documentation	NFR6	28.09.–11.10.
<i>Milestone M3: final portfolio submitted on 11.10.2026</i>				

WP8 ran alongside WP4 to WP7 rather than after them, because a test written in the same increment as its code is what makes each increment deliverable on its own ([Khorikov, 2020](#)). WP9 came first in the final phase because it was the only package depending on an external service, so any delay could still be absorbed before deployment.

The three milestones were fixed by the portfolio schedule, and each was followed by tutorial feedback. The plan therefore reserved time after each milestone to act on that feedback before new work started: after M1 the requirements were extended by FR13 and the rule authoring was specified, and after M2 the architecture, the test cases and the technical debt were reworked, as recorded in Table 1. Treating feedback as planned work rather than as an interruption kept the schedule realistic ([Ambler and Lines, 2022](#)).

2.3 Kanban in GitHub Projects

The project followed Kanban. Waterfall was rejected because it delivers working software only at the end and leaves no room to act on tutorial feedback; Scrum was rejected because its events and roles

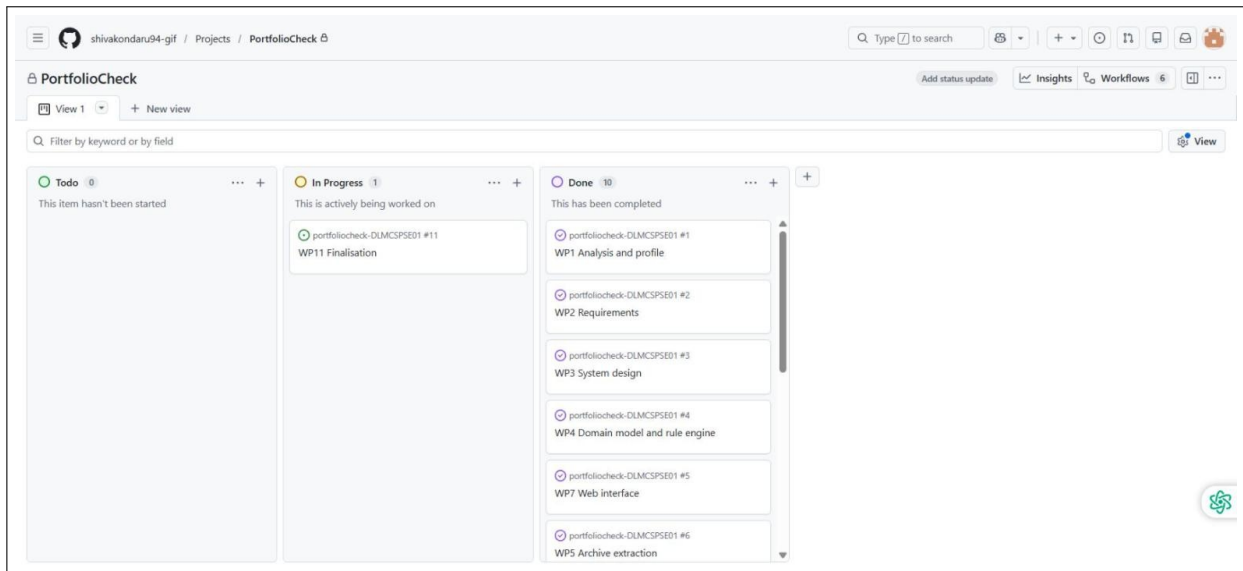


Figure 1: Kanban board in GitHub Projects with the work packages WP1–WP11

presuppose a team ([Schwaber and Sutherland, 2020](#)). Kanban keeps what produces the benefit — a prioritised backlog, limited work in progress and regular review — without the ceremony ([Ambler and Lines, 2022](#)).

In the first two phases the work packages were tracked on a drawn board, which the Phase 2 feedback rightly criticised: a drawing records intentions, not what actually happened. In the finalisation phase the board was therefore moved into GitHub Projects, attached to the project repository ([GitHub, 2025b](#)). Each work package of Table 2 is now an issue in the repository whose description states its phase, its deliverable, the requirements it delivers and its planned dates, and all eleven issues are collected in the project *PortfolioCheck*, shown as a board with the columns *Todo*, *In Progress* and *Done*. The work packages already completed, WP1 to WP9, were entered as issues and closed; GitHub’s built-in project workflow moves a closed issue to *Done* automatically. WP10, the cloud deployment, was then worked through on the board and closed once the service ran live (Section 4.4); WP11, the finalisation, remains in progress until submission. From this point on the board is the working record: an item moves to *In Progress* when work on it starts, and it is closed — and thereby moved to *Done* — only once its tests pass and it has been checked against its requirements ([Kim et al., 2021](#)).

Because every card names the requirements it delivers, the board can be read directly against Table 2 and Table 3, which resolves the inconsistency between the drawn board and the work-package table pointed out in the Phase 2 feedback ([GitHub, 2025b](#)). Figure 1 shows the board.

2.4 Risks

The risk register set up in the conception phase was revisited at every milestone, because a register is only useful if it is kept current rather than written once ([Sommerville, 2020](#)). Of its seven risks, four were closed by the end of the project: scope creep was held off by a fixed minimum rule set, ambiguity in the brief by recording the source provision of every rule, a late deployment failure by containerising from the first increment, and the capacity risk of a single developer by keeping every increment deliverable on its own. The other three remain mitigated rather than closed: malicious uploads (R3), a missed violation that gives false confidence (R6), and an unavailable or rate-limited

GitHub (R7), which degrades the repository check to a warning.

R3 and R6 shaped the design most. R3 exists because accepting an arbitrary archive from the internet is the one genuinely hostile interaction in the system; its mitigation became two security requirements and a separate component. R6 is specific to this kind of tool: a checker that misses a violation is worse than no checker, because it turns a student's justified doubt into false confidence. Its mitigation is a rule of the project rather than of the code — no rule is accepted until a test proves that it rejects a submission violating it (Khorikov, 2020).

3 Requirements and Architecture

3.1 Requirements

The requirements were derived directly from the assignment brief: each formal provision became one verifiable rule, and each rule records the provision it comes from, so that it can be checked against the original — the traceability from requirement to source that requirements engineering demands (Sommerville, 2020). Non-functional requirements follow the quality characteristics of ISO/IEC 25010 (ISO/IEC, 2023). Table 3 gives the final requirements with the test or test case that verifies each one, which turns the table into a record of what the system has been shown to do (Khorikov, 2020).

Table 3: Requirements with priority and verifying evidence

ID	Requirement	Prio.	Verified by
Functional requirements — all implemented			
FR1	Accept a submission archive uploaded in a web browser	Must	Interface tests; M01
FR2	Check the main directory name against the pattern (rule R01)	Must	Rule tests; M02, M04
FR3	Check presence and exact names of the four subdirectories (R02)	Must	Rule tests; M03
FR4	Check that PDF files are genuine PDFs, by content (R05)	Must	Rule test; M05
FR5	Check that no file exceeds the size limit (R06)	Must	Rule test
FR6	Check code archive and links file for phase 3 (R03, R04)	Must	Rule tests; M06, M07
FR7	Report a verdict <i>passed</i> , <i>to review</i> or <i>must fix</i> per rule	Must	Interface tests; all M cases
FR8	Give a corrective action with every non-passing verdict	Must	Enforced by the domain model
FR9	Evaluate only the rules of the selected phase	Should	Phase tests
FR10	Offer a correctly structured empty folder to download	Should	Interface test
FR11	Show the provision of the brief behind each rule	Should	Provenance tests; Figure 4
FR12	Compare the code archive with the public repository (R07)	Could	23 repository tests; M10–M13
FR13	Show the complete rule catalogue before upload	Should	Catalogue test; Figure A.1
Non-functional requirements (ISO/IEC 25010)			
NFR1	<i>Security</i> . Reject absolute paths and path traversal before extraction	—	Extraction tests; M08
NFR2	<i>Security</i> . Reject archives exceeding size, entry or ratio limits	—	Extraction tests
NFR3	<i>Security</i> . Delete every upload before responding; store nothing	—	Code review
NFR4	<i>Security</i> . Run unprivileged on a read-only filesystem	—	Configuration review
NFR5	<i>Performance</i> . Report within 3 seconds for a typical submission	—	Timing test
NFR6	<i>Usability</i> . Report understandable without the brief	—	Manual cases M01–M09
NFR7	<i>Maintainability</i> . A new rule changes only the rule set and its test	—	Adding R07 in phase 3
NFR8	<i>Reliability</i> . A failing rule does not stop the others	—	Resilience test
NFR9	<i>Portability</i> . Deployable with one container command	—	Cloud deployment

Rules are written as code by the maintainer rather than entered by users, which answers the question raised in the Phase 1 feedback. A rule needs logic — matching a pattern, reading a file header, comparing hashes — that no form field can express, and building a rule language with its own editor would have cost as much as the rest of the project for a flexibility nobody in the target group needs. A new rule is one class that declares its identifier, phases and source provision, registered by a decorator and delivered together with a test; it then appears automatically in the rule catalogue of FR13 ([Percival and Gregory, 2020](#)).

Priorities follow the MoSCoW scheme. The *Must* requirements are the provisions every submission has to meet and without which the tool has no value; the *Should* requirements make it convenient and trustworthy; FR12 was marked *Could* because it depends on an external service and was not allowed to endanger the core. The requirements changed in three traceable ways across the phases: FR13 was added after the Phase 1 feedback, FR12 moved from planned to implemented in the final phase, and the verification column was sharpened from a general method to the concrete test that provides the evidence ([Sommerville, 2020](#)).

3.2 Glossary

The glossary fixes the meaning of the domain terms used in this document and in the code; the entries are in alphabetical order ([Sommerville, 2020](#)).

Blob hash	The fingerprint git computes from a file's content; equal hashes mean identical content.
Corrective action	The instruction attached to a non-passing finding that says what must be changed.
Finding	The verdict of one rule on one submission: severity, message, corrective action and source.
Main directory	The single top-level folder of the submission archive.
Phase	One of the three portfolio stages; decides which rules apply.
Report	All findings for one submission, with an overall verdict from the most severe finding.
Rule	A verifiable formal requirement derived from one provision of the brief.
Rule registry	The component that holds the active rules and makes them discoverable.
Severity	The class of a finding: <i>passed</i> , <i>to review</i> or <i>must fix</i> .
Submission	All files a student hands in for one phase, supplied as one archive.
Technical debt	A shortcut in the implementation that saved time at the cost of internal quality.

3.3 Architecture Views

The architecture is documented in two UML views, each answering one question: what the system consists of (Figure 2) and where it runs (Figure 3). Presenting an architecture as a small set of views, each readable on its own, is the established way of making it understandable ([Bass et al., 2021](#); [Brown, 2024](#)).

System context. The student is the only user and works in a web browser. The assignment brief is

not called at runtime; it is the source from which the rules were derived. GitHub is the only external system the application calls, and only for rule R07. Stating the system boundary first makes clear what the application is responsible for and what it depends on (Brown, 2024).

Component view. Figure 2 is a UML component diagram: boxes are components, circles are the interfaces a component provides, and dashed arrows show which component uses which interface; every arrow corresponds to an import in the source code (Richards and Ford, 2020). *WebInterface* receives the upload, has it extracted through *IExtraction*, passes it to *ICheckService* and renders the report; it never refers to an individual rule. *CheckService* is the rule engine and takes the applicable rules from the *RuleRegistry*, which defines the rule interface *IRule* that every rule in *RuleSet* implements. Rule R07 also uses *IRepositoryInventory* to fetch the repository’s file list and *IExtraction* to inspect the nested code archive safely. *DomainModel* holds the Submission, Finding and Report types and depends on nothing, so every rule can be tested without starting a server; extraction lives in one component so that all defences against untrusted input are in one place (Percival and Gregory, 2020; Newman, 2021).

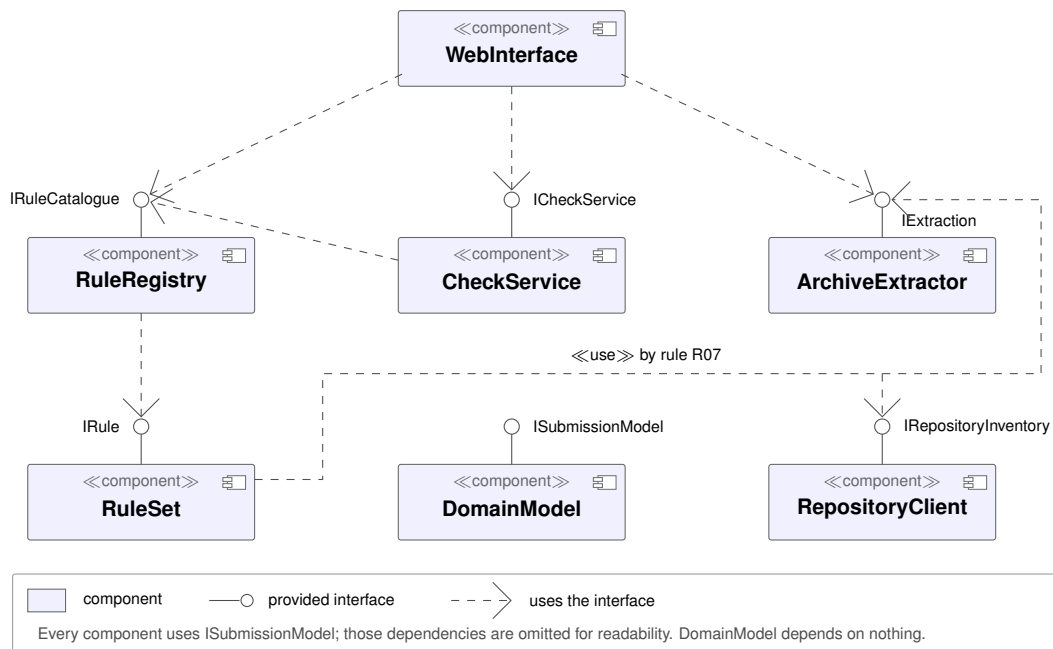


Figure 2: Component view (UML component diagram)

Table 4 maps every component to the module that implements it, and the first line of each module names its component, so the architecture can be found in the code as described (Bass et al., 2021).

Table 4: Components and the source modules that implement them

Component	Source module	Responsibility
WebInterface	app/main.py, app/templates/	Upload, report, rule catalogue, skeleton
CheckService	app/checker.py	Selects, runs and aggregates rules
RuleRegistry	app/rules/base.py	Rule interface and registration
RuleSet	app/rules/naming.py, structure.py, artifacts.py, integrity.py, repository_match.py	Rules R01–R07
DomainModel	app/models.py	Submission, Finding, Report; folder layout
ArchiveExtractor	app/extraction.py	Safety inspection and extraction
RepositoryClient	app/repository.py	GitHub file list and blob hashes

Deployment view. Figure 3 shows where the system runs. The container built from the repository’s Dockerfile runs as a web service on Render; a push to the main branch triggers a new build, and the running container reaches GitHub only through its public API. This view documents the environment that NFR4 and NFR9 constrain ([Render, 2025](#)).

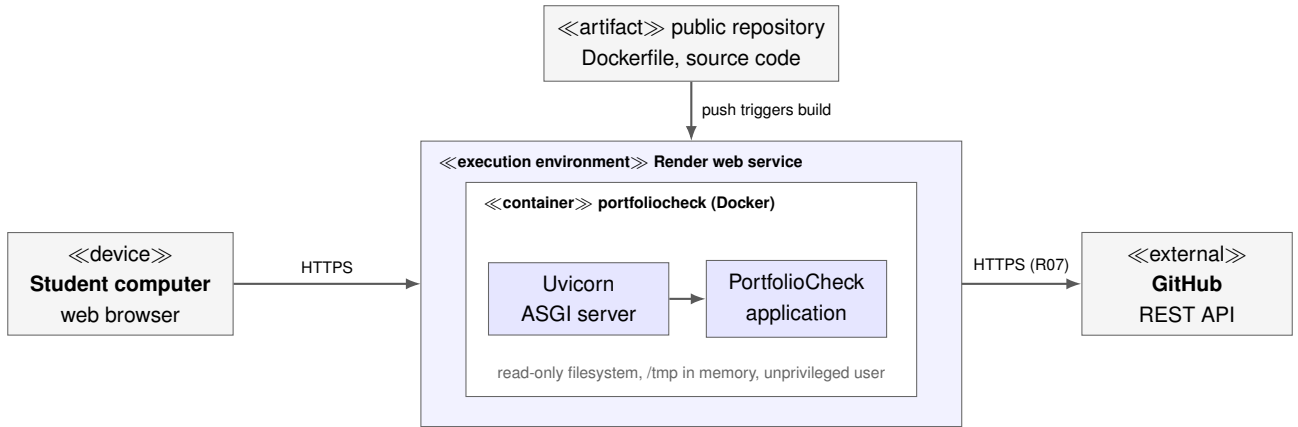


Figure 3: Deployment view (UML deployment notation)

Runtime behaviour. The components cooperate in a fixed order when a submission is checked. WebInterface streams the upload to a temporary folder, stopping as soon as the size limit is reached. ArchiveExtractor inspects the archive against the safety limits and only then extracts it, returning a Submission. CheckService selects the rules of the chosen phase from the registry and runs each one; each rule returns one or more findings, and rule R07 additionally asks RepositoryClient for the repository’s file list. CheckService combines all findings into a Report, WebInterface renders it, and the temporary folder is deleted before the response leaves the server. Describing this order explicitly matters because a structural view alone does not show it ([Bass et al., 2021](#)).

Architectural decisions. Four decisions shaped the architecture more than any other, each accepted with a known cost ([Ford et al., 2021](#)). *Rules as independent units behind a registry*: a rule can be added without touching the engine or the interface, as the addition of R07 in the final phase showed; the cost is the global registry recorded as TD1. *A dependency-free domain model*: every rule can be tested against an in-memory submission in milliseconds; the cost is a small translation step where uploaded files become a Submission ([Percival and Gregory, 2020](#)). *No persistence*:

NFR3 holds by construction and no student work is ever stored; the cost is that no history of checks can be offered, which no requirement asked for (OWASP, 2021). *The external service behind an interface that fails softly*: the core check never depends on GitHub and R07 can be tested offline; the cost is that an offline student gets an incomplete answer for one rule, which the report states (Newman, 2021).

Cross-cutting concepts. Some concerns apply to every component and are handled in one consistent way (Bass et al., 2021). *Error handling* follows two rules: an input that cannot be processed, such as a corrupt or unsafe archive, is rejected with an explanatory page before any rule runs, while a failure inside one rule is caught by the engine, logged and reported as a warning for that rule alone (NFR8). *Security* is concentrated at the boundary, where every archive — including the code archive nested inside it — passes the same inspection. *Configuration* is limited to two environment variables, the port and an optional GitHub token, so one container image serves every environment. *Traceability* is carried by the data itself: every finding holds its rule identifier and the provision of the brief it comes from.

3.4 Technology Stack

The application is written in Python 3.11, because archive handling and hashing are built into its standard library (Python Software Foundation, 2025). It uses FastAPI as web framework, Jinja templates for the two pages, Uvicorn as server and pytest for the tests (Ramírez, 2025; Pallets, 2025b; Encode, 2025; pytest-dev, 2025); it runs as a Docker container on Render (Docker, 2024; Render, 2025). There is no database, because NFR3 forbids storing submissions, and GitHub is read with the standard library rather than a client package, because two read-only requests do not justify a dependency. Recording the rejected options with each decision keeps it reviewable later (Ford et al., 2021); all libraries are open source under the MIT, BSD or Apache licence.

The web framework was the closest decision. Flask and FastAPI are both small and would have served; FastAPI was chosen because it validates the request at the boundary, which is exactly where untrusted uploads arrive, and because it runs blocking work in a thread pool without extra code (Ramírez, 2025; Pallets, 2025a). Django was rejected not because it is weaker but because most of what it offers — a database layer, migrations, an administration interface — would have remained unused in a system that stores nothing (Django Software Foundation, 2025). The same reasoning ruled out a single-page front end: the interface is one form and one report, and a separate JavaScript application would have added a second build, a second deployment artifact and an interface contract to maintain (Richards and Ford, 2020).

4 Implementation and Evaluation

4.1 Implementation

The implementation grew outward from the centre of the component diagram. The domain model came first, and one decision there shaped everything else: a finding worse than *passed* cannot be created without a corrective action, so FR8 is enforced by the type itself rather than by the care of whoever writes the next rule (Percival and Gregory, 2020). Archive extraction followed, deliberately before any rule, so that no untrusted archive ever passed through an unprotected path: an archive is rejected before anything is written to disk if it contains absolute paths or `../` segments, more than 5,000 entries, more than 200 MB of content, or an entry compressed more than 200:1 — the signatures of path traversal and decompression bombs (OWASP, 2021; NIST, 2022).

The rules were then added one at a time, each with its test, and each designed around the mistake students actually make (Combéfis, 2022). *R01* checks the main folder name against the pattern `Surname-FirstName_MatrNo_Course`, accepts German umlauts in names, and only warns when the matriculation number has an unusual length, since that may be correct. *R02* checks the four required subfolders; when one is missing it looks for a folder that differs only in case or separators, such as `02_requirements`, and names it, because the common mistake is a near miss rather than an omission. *R03* looks for the code archive in `04-Implementation` and checks its name. *R04* reads every `.txt` file for a GitHub address and warns if the address of the deployed application is missing. *R05* reads the first bytes of every file ending in `.pdf` instead of trusting the extension, because a Word file saved under a PDF name is exactly the failure an examiner would meet. *R06* checks the 15 MB limit and names the largest file. Rules *R03*, *R04* and *R07* apply only to phase 3, where these artifacts are required.

The web interface came last. It discovers the rules through the registry, so the rule catalogue on the start page stays correct as rules are added, and it runs extraction and rule checks in a worker thread, so a slow check does not block other users (Ramírez, 2025).

The last and most distinctive rule, *R07*, checks that the public repository and the submitted code archive are identical. It reads the repository address from the links file, asks the GitHub REST API for the repository's complete file tree with the *blob hash* of every file (GitHub, 2025a), and computes the same hash for every file in the archive, so content can be compared without downloading anything (Chacon and Straub, 2024). Windows line endings, the wrapper folder added by GitHub's *Download ZIP*, and macOS or `.git` metadata are ignored so that they cause no false alarms. If GitHub cannot be reached, *R07* gives a warning and all other rules still run; a private or missing repository is a failure, because the brief requires it to be public. The code archive nested inside the submission is untrusted in the same way as the upload itself, so it passes the same safety inspection before any of its files is read (OWASP, 2021). The result names every file that exists only in the repository, only in the archive, or in both with different content, so the student knows exactly what to fix.

The finished application consists of 1,136 lines of Python in twelve modules and is accompanied by 896 lines of tests. Every module opens with a line naming the architectural component it implements; public functions carry type annotations; and comments explain why a decision was taken rather than repeating what the code does. A static analysis with `pyflakes` reports no defects other than the deliberate registry imports recorded as TD1. Code that is small, typed and self-explaining matters most in a project that will be read by someone other than its author, such as an examiner (Winters et al., 2020).

4.2 Testing

Testing combined manual test cases with an automated suite. Table 5 specifies each manual test case by its exact input and expected behaviour, as the Phase 2 feedback requested; specifying tests this way makes them repeatable by someone other than their author (Winters et al., 2020). Inputs M01–M09 are realistic sample submissions for a fictional project, generated by a script, each differing from the compliant one in exactly one respect so that each case tests exactly one rule; they use the example name from the brief so that no real person's data appears in the repository (NIST, 2022). Cases M10–M13 use the real project repository. Every case is run by choosing the file and phase on the start page and pressing *Check submission*; Appendix A shows the actual output of the application

for each of the cases M01–M09.

Table 5: Manual test cases with input, expected behaviour and result

ID	Input	Ph.	Expected behaviour	Result
M01	M01: compliant submission	1	<i>Ready to submit</i> ; R01, R02, R05, R06 pass	Pass
M02	M02: main folder Mustermann_Max_12345678_PSE	1	R01 fail: <i>does not follow the required pattern</i>	Pass
M03	M03: folder 02_requirements	1	R02 fail: <i>A folder named '02_requirements' exists</i>	Pass
M04	M04: matriculation number 123456	1	<i>Almost there</i> ; R01 warn: <i>is not 8 digits long</i>	Pass
M05	M05: Word file saved as architecture.pdf	2	R05 fail: <i>has a .pdf extension but is not a PDF file</i>	Pass
M06	M06: code archive named code.zip	3	R03 fail: <i>No correctly named code archive</i>	Pass
M07	M07: links file without application URL	3	R04 warn: <i>contains no second URL for the deployed application</i>	Pass
M08	M08: entry ../outside-the-upl oad-folder.txt	any	Error page: <i>Unsafe path in archive</i> ; nothing extracted	Pass
M09	M09: text file renamed to .zip	any	Error page: <i>not a valid ZIP archive</i>	Pass
M10	Phase 3 submission; code archive made from the repository with git archive	3	R07 pass: <i>All N files ... are identical</i>	Pass
M11	As M10, one line of app/main.py changed in the code archive only	3	R07 fail naming app/main.py	Pass
M12	As M10, links file naming a repository that is not publicly visible	3	R07 fail: <i>does not exist or is not public</i>	Pass
M13	As M10, GitHub unreachable (connection refused)	3	R07 warn: <i>GitHub could not be reached</i>	Pass

Cases M10 to M13 test R07 against the real repository and were run after deployment, M10 to M12 on the deployed application itself. For M10, the code archive was created from the current state of the repository, and the check confirmed that all 66 files are identical. M11 changes one line of a source file in the archive only, and the check named exactly that file. M12 names a repository address that GitHub does not serve publicly; GitHub answers a private and a non-existent repository with the same response, so this case covers both. M13 was run locally with every connection to GitHub refused; R07 degraded to a warning while the other six rules still produced their findings. Together these four cases exercise every outcome of R07 on real data rather than on the simulation used in the automated tests (Winters et al., 2020).

The test data was designed with three properties. Each sample differs from the compliant one in exactly one respect, so a failure can only have one cause. The samples are generated by a script rather than assembled by hand, and the script fixes all timestamps, so every run produces byte-identical archives that can be reviewed and recreated. And the documents inside the samples are valid PDF files with meaningful text about a fictional study-room booking project, as the brief requires sensible test data rather than placeholder content. Test data that is realistic, minimal in its variation and itself under test is what makes a test result convincing (Khorikov, 2020).

The automated suite contains 73 tests in five modules. The 23 *rule tests* exercise every rule against a compliant submission and against one that violates exactly that rule; they also include a resilience test, in which a deliberately broken rule is added and the remaining rules must still produce their findings (NFR8), and a timing test for NFR5. The 5 *extraction tests* feed crafted archives to every safety limit: a path with `../`, an absolute path, 5,001 entries and a 50 MB file compressed to a few kilobytes. The 23 *repository tests* check R07 against a simulated GitHub that can be told to succeed, to report a missing repository, to hit the rate limit or to be unreachable, so no test depends on the network; three of them confirm that the computed hashes equal those of `git hash-object`. The 10 *interface tests* exercise the whole path from upload to report, including the error pages, and the 12 *sample tests* confirm that every input of Table 5 produces the documented result (Khorikov, 2020). The last group ties the manual and the automated tests together: the expected results in Table 5 are themselves executed on every run, so the specification cannot drift away from the behaviour of the application. Figure A.13 in Appendix A shows a complete run in which all 73 tests pass; the suite was also run from a clean extraction of the submission archive to confirm that the published code is self-contained (pytest-dev, 2025).

4.3 Results

Figure A.1 in Appendix A shows the start page of the running application. It offers the upload form and the phase selection, and it lists every rule with the provision of the brief behind it, so a student knows exactly what will be checked before uploading anything (FR13). The link below the form downloads an empty, correctly named folder structure for students starting from nothing (FR10) (Sommerville, 2020).

Figure 4 shows the report for sample M05 checked as phase 3. The headline says the submission is not ready and counts three problems to fix, one to review and three passed rules. The Word file saved as `architecture.pdf` is caught by R05; because the sample is a phase-2 folder, the code archive and links file that phase 3 requires are missing, so R03 and R04 fail, and R07 explains that the repository comparison could not run until those are fixed. Failures come first, each with its correction and the chapter of the brief it derives from (FR8, FR11), which is the behaviour the project set out to deliver (Messer et al., 2024).

Measured against the four objectives of Section 2.1, the result is as follows. Every formal provision of the brief that can be checked mechanically is covered by one of the seven rules; the only provisions left out are the typographic rules for the abstract, which would require parsing page layout. Every failure carries its correction, which the domain model enforces. Without the GitHub comparison a typical submission is checked in well under a second, as the timing test confirms; in phase 3 the two GitHub requests are bounded by a timeout of 2.5 seconds, after which R07 reports a warning instead of waiting. And the application is reachable in a browser without installation or login (Sommerville, 2020).

All thirteen functional requirements are implemented, including the optional FR12, and each is verified by an automated test and, where visible in the interface, by a manual test case (Table 3). Of the non-functional requirements, NFR1, NFR2, NFR5 and NFR8 are verified by automated tests, NFR3 and NFR4 by review of the code and the container configuration, NFR6 by the manual test cases, NFR7 by the addition of R07, which needed a new rule module and a new client module but no change to the engine or the interface, and NFR9 by the cloud deployment (ISO/IEC, 2023).

PortfolioCheck

Check your portfolio submission against the formal requirements — before you hand it in.

Not ready to submit

M05-phase2-renamed-docx.zip · checked as phase 3

3 must fix 1 to review 3 passed

Findings

R03

Zipped code in 04-Implementation

FAIL

No ZIP file found in 04-Implementation.

How to fix: Zip your complete program code and place it in 04-Implementation as Surname-FirstName_MatrNo_Course_Submission_Code.zip.

Source: Chapter 4.2: in 04-Implementation you upload a zipped version with the naming convention Name-FirstName_MatrNo_Course_Submission_Code.zip

R04

GitHub link in .txt file

FAIL

No GitHub repository URL found in any .txt file (checked: 'README.txt').

How to fix: Write the full URL in the form `https://github.com/<user>/<repository>` into the .txt file.

Source: Chapter 4.1: a .txt file including the link to your public GitHub repository, and -- for a web application -- the link to the application

R05

PDF files are real PDFs

FAIL

'Mustermann-Max_12345678_PSE/03-Architecture-documentation/architecture.pdf' has a .pdf extension but is not a PDF file.

How to fix: Export the document to PDF properly instead of renaming it; a renamed .docx cannot be opened by the examiner and therefore cannot be assessed.

Source: Chapter 4.2: permitted file formats -- PDF

R07

GitHub repository matches code archive

WARN

The comparison could not run because the GitHub link (R04) was not found.

How to fix: Fix the findings for R03 and R04 first; this check then runs automatically.

Source: Chapter 1.1.3: the content of the GitHub repository and the contents of the ZIP file uploaded to your zip folder should be identical

R01

Main directory name

PASS

Main directory 'Mustermann-Max_12345678_PSE' follows the required pattern.

Source: Chapter 4.2: Main directory (name of the zip folder) -> Name-First_Name_Matriculation_Course

R02

Required subdirectories

PASS

All four required subdirectories are present.

Source: Chapter 4.2: subdirectories 01-Project-management ... 04-Implementation

R06

File size limits

PASS

All files are within the 15 MB limit (largest: requirements.pdf, 0.0 MB).

Source: Chapter 4.2: file size max. 15 MB (conception and development phase)

Figure 4: Report for sample M05 checked as phase 3

The tool also has limits, stated here rather than left for users to discover. Its rules encode the brief of one course; the PDF rule confirms that a file is a PDF but not that it opens cleanly; the repository check depends on GitHub; and the tool checks form, not content, so a submission that passes every rule can still fail its assessment. The interface says so on every page, because a compliance tool mistaken for a quality tool would do more harm than good (Messer et al., 2024).

4.4 Deployment and Installation

The application runs as a Render web service built from the repository's Dockerfile and blueprint file; the port is taken from the environment, so the same image runs locally and in the cloud (Render, 2025; Docker, 2024). No login is required: a student opens the address, chooses the zip folder and the phase, and presses *Check submission*. The free tier pauses the service after a period of inactivity, so the first request afterwards can take up to a minute; this is noted in the links file for the examiner.

For local use the README in the repository gives two procedures. With Docker, a single command, `docker compose up -build`, builds and starts the container, which then serves the application at `http://localhost:8000`. Without Docker, a Python 3.11 virtual environment is created, the dependencies are installed from `requirements.txt`, and the server is started with `uvicorn app.main:app`; the tests run with `python -m pytest`. An optional `GITHUB_TOKEN` environment variable raises the GitHub rate limit used by R07. Keeping installation to one command is what makes the application usable by an examiner without assistance (Kim et al., 2021).

- **Application:** <https://portfoliocheck-bwwq.onrender.com>
- **Repository:** <https://github.com/shivakondaru94-gif/portfoliocheck-DLMCSPSE01>

Figure A.12 in Appendix A shows the service running live on Render.

5 Reflection

5.1 Technical Debt

Technical debt means shortcuts in the implementation that saved time at the cost of internal quality and that, like financial debt, should be repaid before the interest grows (Avgeriou et al., 2023). It is not a list of missing features; all functional requirements are implemented. Table 6 lists the debt that remains, ordered by its effect on maintenance (Lenarduzzi et al., 2021).

Table 6: Open technical debt

ID	Shortcut	Consequence	Repayment
TD1	Rule registry is a global list filled by import side effects	A rule module not imported is silently inactive; tests must patch shared instances	Explicit registry object created at start-up
TD2	Limits hard-coded as constants in four modules	Changing a limit needs a code change and redeployment	One settings object read from the environment
TD3	GitHub responses not cached	Without a token, about 30 phase-3 checks per hour before R07 degrades	Short-lived cache per repository
TD4	PDF check reads only the file header	A corrupt PDF with a valid header passes R05	Parse each PDF with a PDF library
TD5	R07 repeats the link and archive lookup of R03 and R04	The three rules could disagree after one is changed	Shared lookup functions
TD6	Interface tests check HTML text	A purely visual change can break tests	JSON report for tests
TD7	Messages written as English strings inside rule classes	Rewording or translating means editing rule logic	Message catalogue

The order of the table reflects the cost of leaving each item in place. TD1 and TD2 come first because they affect every future change: any new rule depends on the registry, and any new deployment depends on the limits. TD3 matters as soon as the application is shared by many students before a deadline. TD4 to TD7 are local and can be repaid independently when the affected part is next changed. This order follows the common practice of prioritising debt by how much further work it makes more expensive ([Lenarduzzi et al., 2021](#)).

Three items were repaid during the project: a duplicated regular expression was moved into a shared module, blocking work on the request thread was moved to a worker thread before R07 added network calls, and the untested error path of the rule engine received a test ([Winters et al., 2020](#)).

5.2 Lessons Learned

Tools. Treating the standard library as the first choice kept the code that handles untrusted input free of third-party dependencies ([Python Software Foundation, 2025](#)), and git’s own content hashes proved the right way to compare repository and archive; comparing file sizes would have missed edits that keep a file’s length ([Chacon and Straub, 2024](#)). The Kanban board, by contrast, was first drawn instead of kept in a tool, and the Phase 2 feedback showed why that was wrong: a drawing shows intentions, while a board linked to commits shows what happened ([GitHub, 2025b](#)).

Method. Working on one work package at a time mattered more than expected, because it stopped the rule set from growing faster than its tests ([Ambler and Lines, 2022](#)). Writing each test together with its rule meant that the late addition of R07 could not break R01–R06 unnoticed, and building the extraction defences before any rule meant they never had to be added afterwards ([Khorikov, 2020](#)). The tutorial feedback proved as valuable as the development loop itself: it corrected the architecture notation, the meaning of technical debt and the way test cases are written ([Schwaber and Sutherland, 2020](#)).

Time. The single external dependency, R07, took the largest share of the final phase, as the risk register had predicted. Isolating it behind an interface early meant the rest of the system never waited for it and that it could be tested fully without the network, and scheduling it first in the phase

left time for deployment ([Sommerville, 2020](#)).

What would be done differently. Three things would change in a second attempt. The board would live in a tool from the first day, because its evidence only accumulates while it is in use. The rule registry would be an explicit object from the start; the global list was quicker to write but is now the first item of technical debt, and replacing it later costs more than building it properly would have. And the manual test cases would be written in their final, concrete form during the conception phase rather than as general test levels, since concrete inputs and expected outputs expose unclear requirements earlier ([Avgeriou et al., 2023](#); [Winters et al., 2020](#)).

5.3 Conclusion

PortfolioCheck addresses a narrow and practically unaddressed problem: formal submission requirements are mechanical and carry marks, yet they are checked by hand at the least favourable moment. The finished application checks seven rules derived from the brief, explains every failure with its correction and source, and verifies that repository and code archive are identical — a provision that cannot practically be checked by hand. All functional requirements are implemented, every quality requirement has recorded evidence, and the architecture documented here is the one found in the code ([Bass et al., 2021](#)). The most useful next steps follow from the technical debt: an explicit registry and external configuration would let the rules be adapted to other courses, and a cache for repository data would let one deployment serve a whole cohort in the days before a deadline ([Ford et al., 2021](#)).

References

- Ambler, S. W. and Lines, M. (2022) *Choose Your WoW! A Disciplined Agile Approach to Optimizing Your Way of Working*. 2nd edn. Newtown Square, PA: Project Management Institute.
- Avgeriou, P., Ozkaya, I., Chatzigeorgiou, A., Ciolkowski, M., Ernst, N. A., Koontz, R. J., Poort, E. and Shull, F. (2023) 'Technical debt management: the road ahead for successful software delivery', in *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, pp. 15–30.
- Bass, L., Clements, P. and Kazman, R. (2021) *Software Architecture in Practice*. 4th edn. Boston, MA: Addison-Wesley.
- Brown, S. (2024) *The C4 Model for Visualising Software Architecture*. Available at: <https://c4model.com/> (Accessed: 1 October 2026).
- Chacon, S. and Straub, B. (2024) *Pro Git*. 2nd edn (continuously updated). Available at: <https://git-scm.com/book/> (Accessed: 1 October 2026).
- Comb  fis, S. (2022) 'Automated code assessment for education: review, classification and perspectives on techniques and tools', *Software*, 1(1), pp. 3–30.
- Django Software Foundation (2025) *Django Documentation*. Available at: <https://docs.djangoproject.com/> (Accessed: 1 October 2026).
- Docker Inc. (2024) *Compose Specification*. Available at: <https://docs.docker.com/compose/> (Accessed: 1 October 2026).
- Encode (2025) *Uvicorn: An ASGI Web Server for Python*. Available at: <https://www.uvicorn.org/> (Accessed: 1 October 2026).
- Ford, N., Richards, M., Sadalage, P. and Dehghani, Z. (2021) *Software Architecture: The Hard Parts — Modern Trade-Off Analyses for Distributed Architectures*. Sebastopol, CA: O'Reilly Media.
- GitHub Inc. (2025) *REST API Endpoints for Git Trees*. GitHub Docs. Available at: <https://docs.github.com/en/rest/git/trees> (Accessed: 1 October 2026).
- GitHub Inc. (2025) *About Projects*. GitHub Docs. Available at: <https://docs.github.com/en/issues/planning-and-tracking-with-projects> (Accessed: 1 October 2026).
- Hermans, F. (2021) *The Programmer's Brain: What Every Programmer Needs to Know about Cognition*. Shelter Island, NY: Manning.
- International Organization for Standardization (2023) *ISO/IEC 25010:2023 Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*. Geneva: ISO.
- Khorikov, V. (2020) *Unit Testing Principles, Practices, and Patterns*. Shelter Island, NY: Manning.
- Kim, G., Humble, J., Debois, P., Willis, J. and Forsgren, N. (2021) *The DevOps Handbook*. 2nd edn. Portland, OR: IT Revolution Press.
- Lenarduzzi, V., Besker, T., Taibi, D., Martini, A. and Arcelli Fontana, F. (2021) 'A systematic literature review on technical debt prioritization: strategies, processes, factors, and tools', *Journal of Systems and Software*, 171, 110827.
- Messer, M., Brown, N. C. C., K  lling, M. and Shi, M. (2024) 'Automated grading and feedback tools for programming education: a systematic review', *ACM Transactions on Computing Education*, 24(1), pp. 1–43.
- Newman, S. (2021) *Building Microservices: Designing Fine-Grained Systems*. 2nd edn. Sebastopol, CA: O'Reilly Media.
- National Institute of Standards and Technology (2022) *SP 800-218: Secure Software Development Framework (SSDF) Version 1.1*. Gaithersburg, MD: NIST.

- Open Worldwide Application Security Project (2021) *OWASP Top 10:2021*. Available at: <https://owasp.org/Top10/> (Accessed: 1 October 2026).
- Paiva, J. C., Leal, J. P. and Figueira, Á. (2022) 'Automated assessment in computer science education: a state-of-the-art review', *ACM Transactions on Computing Education*, 22(3), pp. 1–40.
- Pallets Projects (2025) *Flask Documentation*. Available at: <https://flask.palletsprojects.com/> (Accessed: 1 October 2026).
- Pallets Projects (2025) *Jinja Documentation*. Available at: <https://jinja.palletsprojects.com/> (Accessed: 1 October 2026).
- Percival, H. and Gregory, B. (2020) *Architecture Patterns with Python*. Sebastopol, CA: O'Reilly Media.
- pytest-dev Team (2025) *pytest Documentation*. Available at: <https://docs.pytest.org/> (Accessed: 1 October 2026).
- Python Software Foundation (2025) *zipfile — Work with ZIP Archives*. Python 3.11 Standard Library Documentation. Available at: <https://docs.python.org/3/library/zipfile.html> (Accessed: 1 October 2026).
- Ramírez, S. (2025) *FastAPI Documentation*. Available at: <https://fastapi.tiangolo.com/> (Accessed: 1 October 2026).
- Render Services Inc. (2025) *Render Documentation: Docker and Blueprints*. Available at: <https://render.com/docs> (Accessed: 1 October 2026).
- Richards, M. and Ford, N. (2020) *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA: O'Reilly Media.
- Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide: The Definitive Guide to Scrum*. Available at: <https://scrumguides.org/> (Accessed: 1 October 2026).
- Sommerville, I. (2020) *Engineering Software Products: An Introduction to Modern Software Engineering*. Harlow: Pearson Education.
- Winters, T., Manshreck, T. and Wright, H. (2020) *Software Engineering at Google: Lessons Learned from Programming over Time*. Sebastopol, CA: O'Reilly Media.

Appendix A: Working Results

This appendix shows the application at work for the manual test cases of Table 5, so that each result recorded there as *Pass* can be seen. Each screenshot was taken on a laptop of the complete browser window while the application was running locally: the sample submission named in the caption was uploaded through the start page with the phase given in the table, and the address bar shows the page the application returned. For long reports the window shows the top of the page, where the failures are listed first. Case M05 is shown in Figure 4. The phase-3 samples M06 and M07 link to the example repository address from the assignment brief, which does not exist, so rule R07 reports a failure for them, as documented in the repository's sample description. The appendix also shows the start page, the run of the automated test suite and the deployed service.

PortfolioCheck
Check your portfolio submission against the formal requirements — before you hand it in.

Check your submission
Upload the zip folder you are about to submit. Every file stays in memory for the duration of the check and is deleted immediately afterwards.

Your zip folder
 No file chosen

Portfolio phase
All checks (no phase selected) ▼

Starting from scratch? [Download an empty folder structure](#) and fill it in.

What gets checked
Also available as JSON at [/rules](#).

R01	Main directory name Chapter 4.2: Main directory (name of the zip folder) -> Name-First_Name_Matriculation_Course
R02	Required subdirectories Chapter 4.2: subdirectories 01-Project-management ... 04-Implementation

Figure A.1: Start page with upload form and rule catalogue

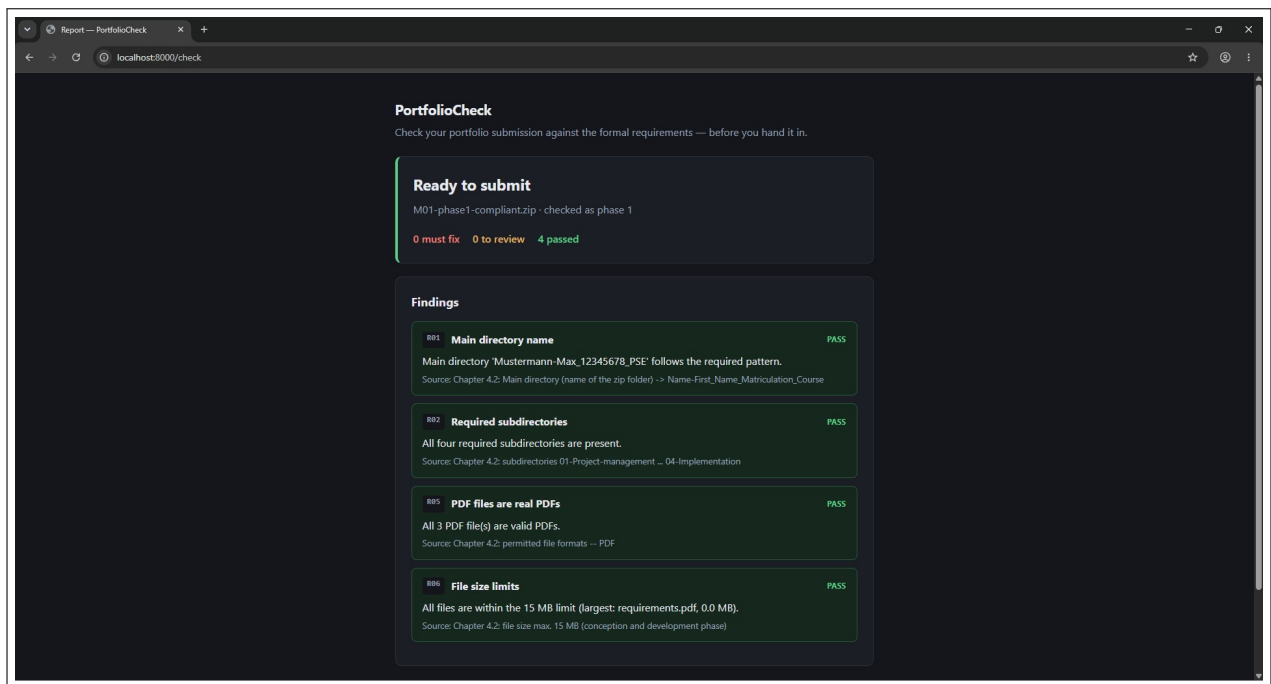


Figure A.2: M01, phase 1: compliant submission — *Ready to submit*

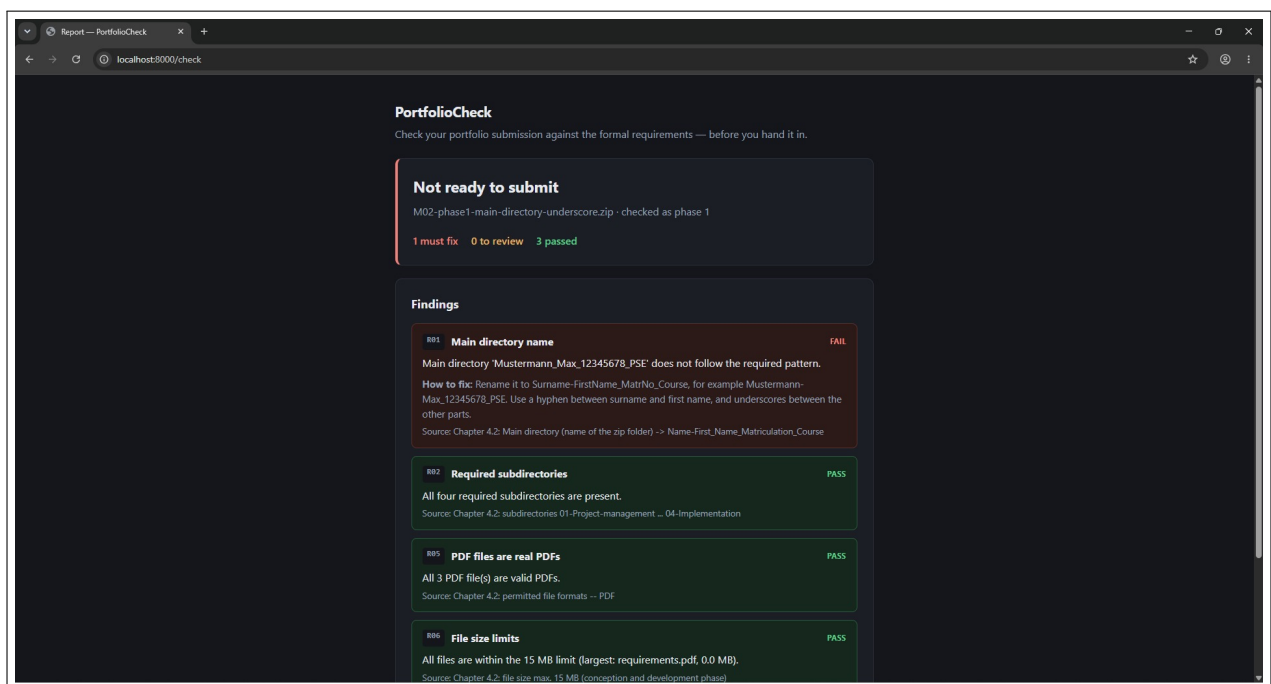


Figure A.3: M02, phase 1: underscore in the main directory name — R01 fails

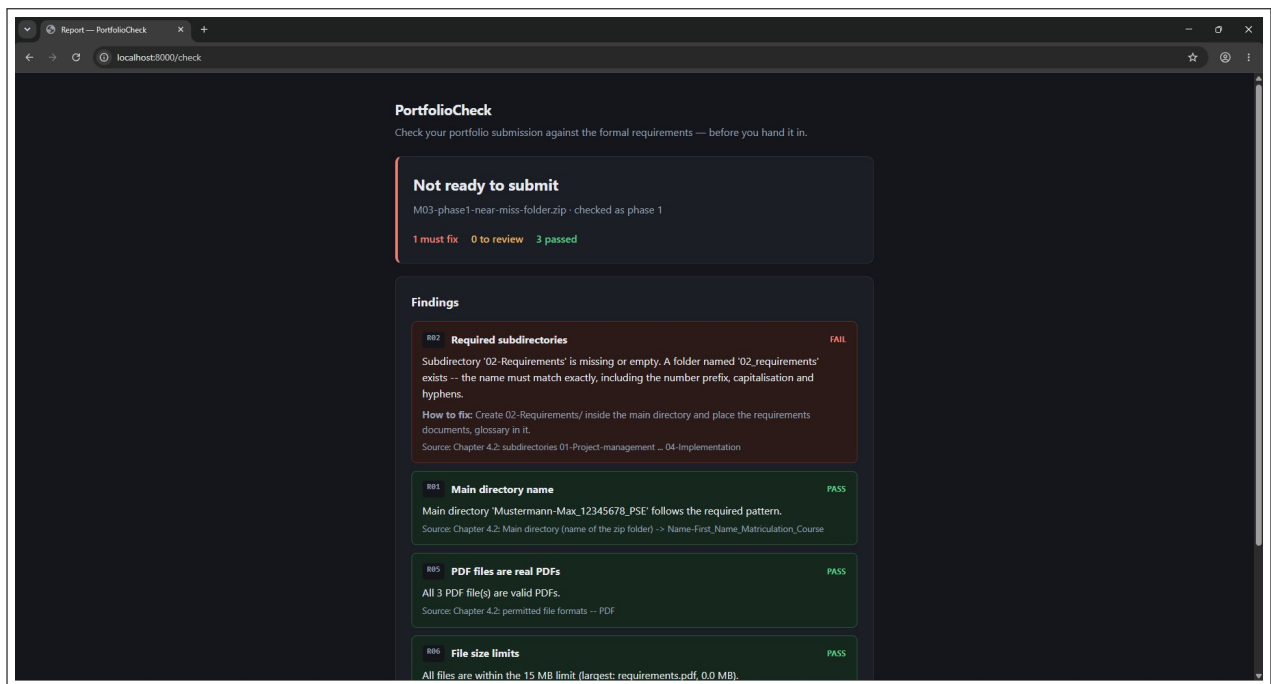


Figure A.4: M03, phase 1: folder 02_requirements — R02 fails and names it

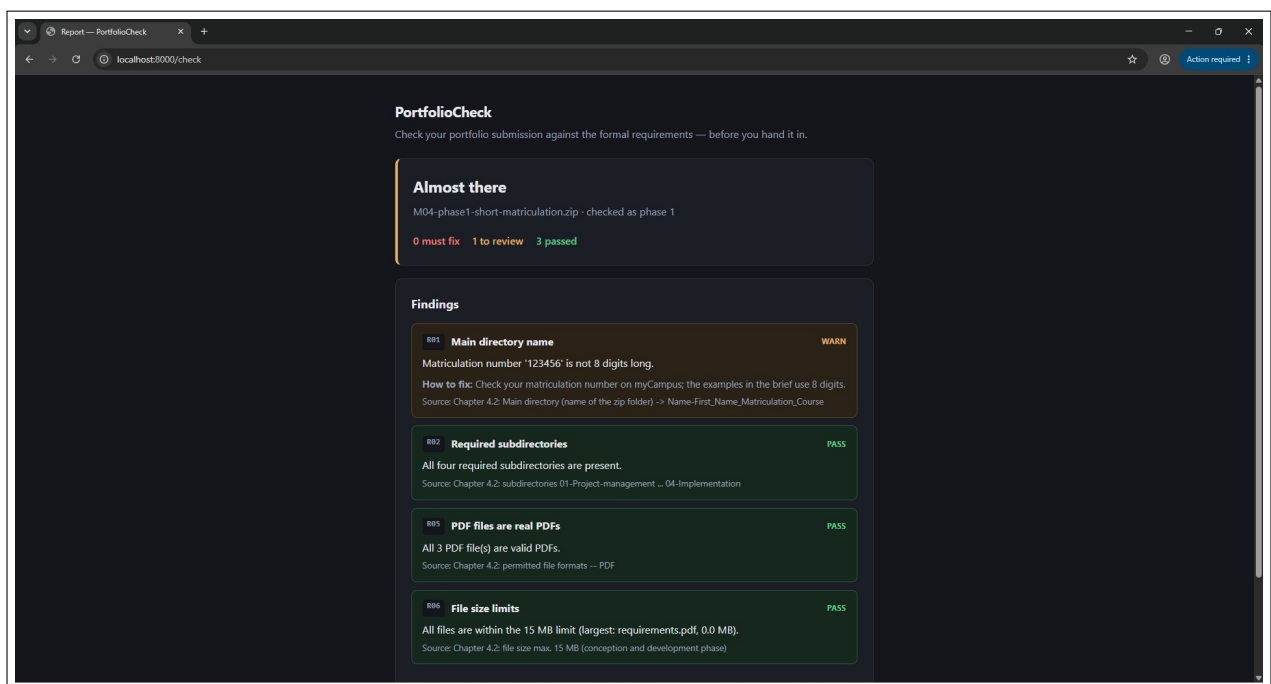


Figure A.5: M04, phase 1: six-digit matriculation number — R01 warns, *Almost there*

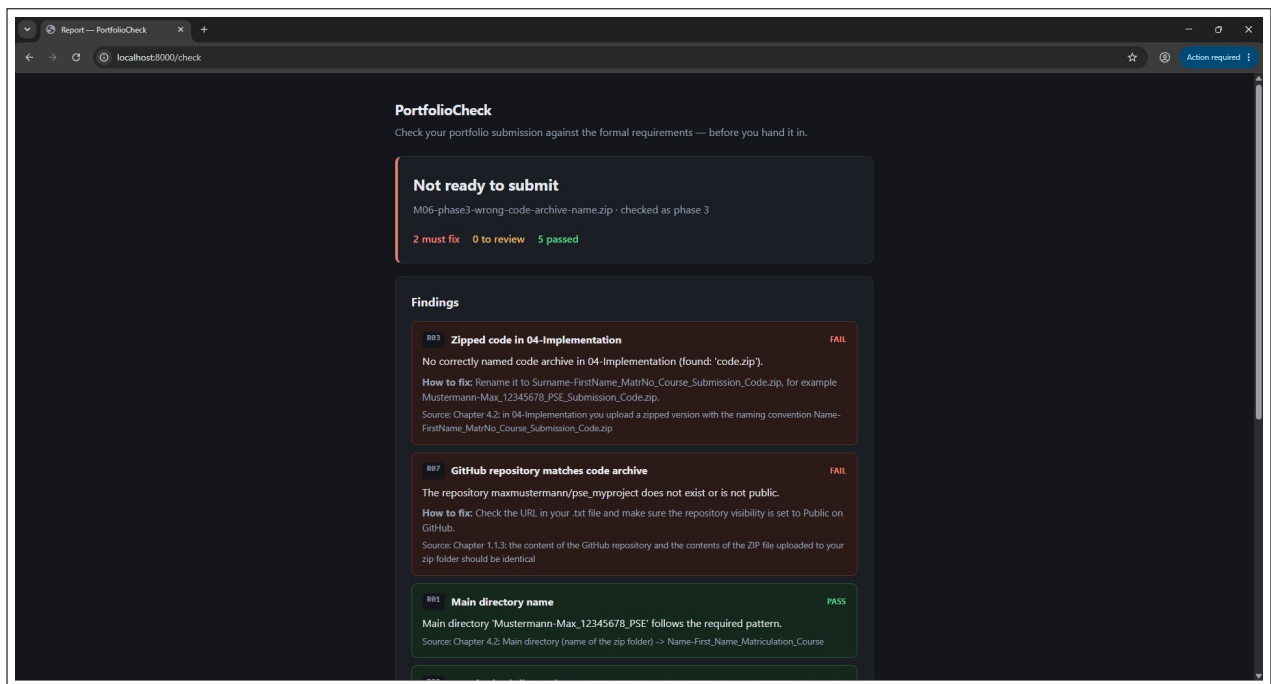


Figure A.6: M06, phase 3: code archive named code.zip — R03 fails

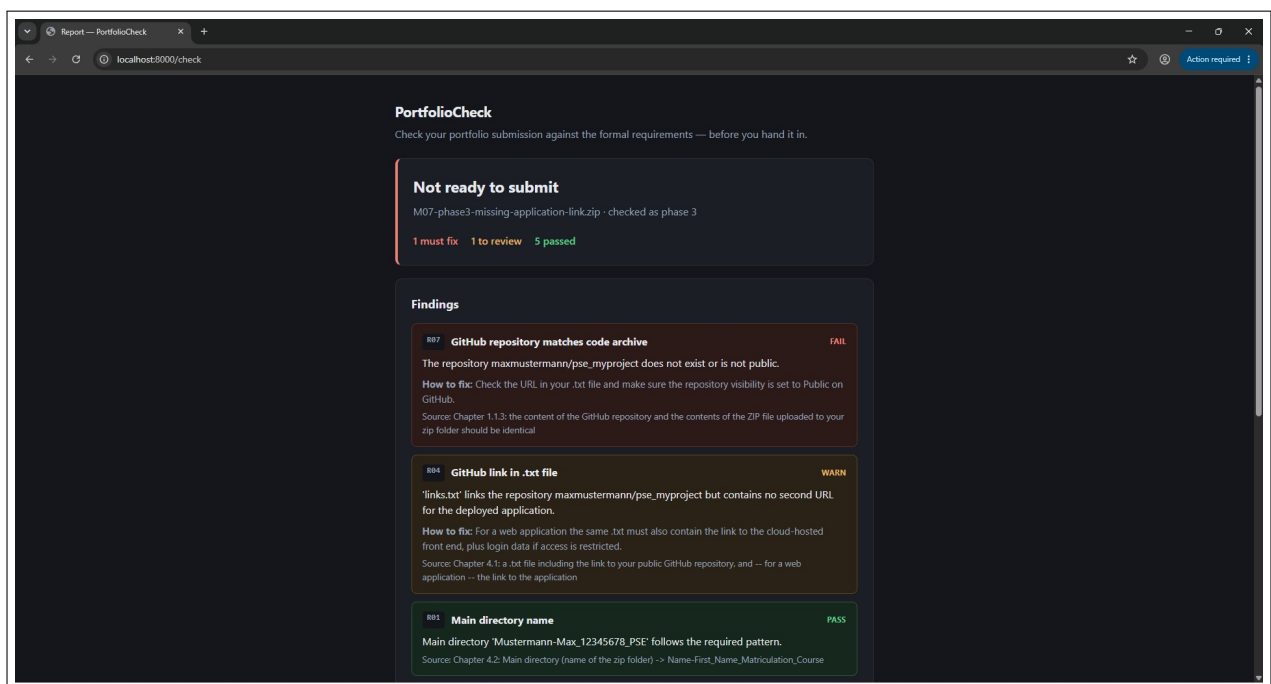


Figure A.7: M07, phase 3: links file without application URL — R04 warns

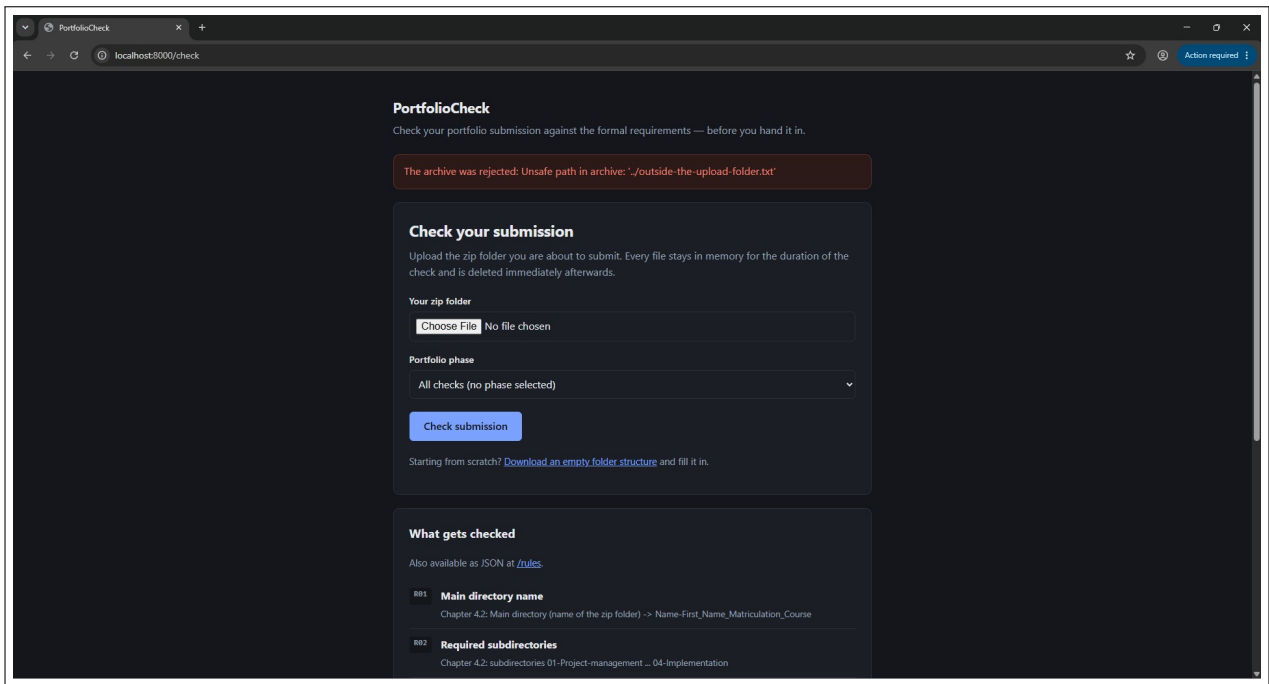


Figure A.8: M08: archive containing a `../` path — rejected before extraction

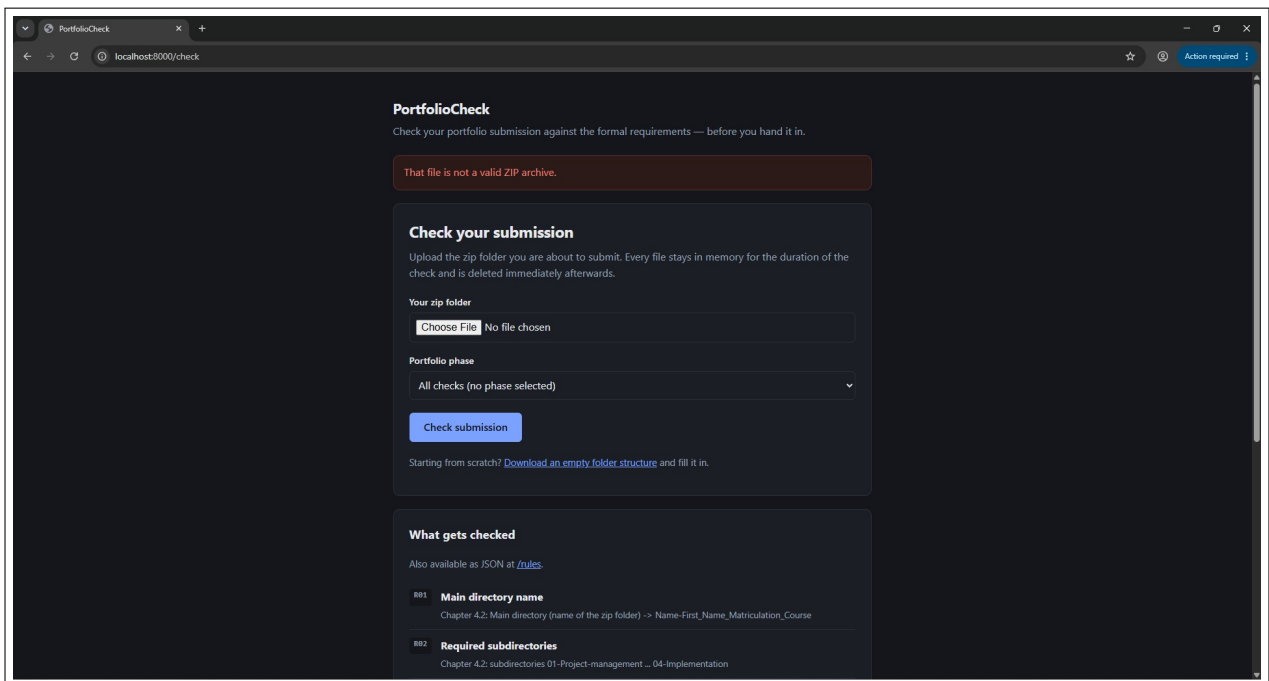


Figure A.9: M09: text file renamed to `.zip` — rejected as invalid

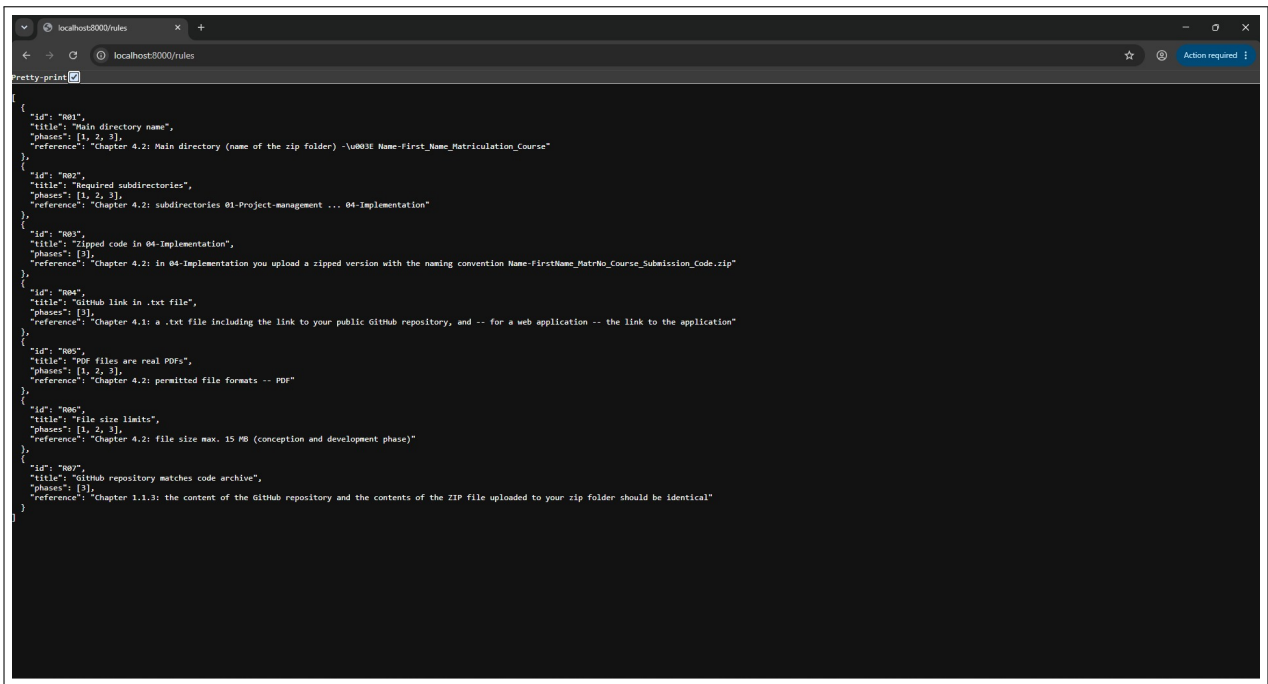


Figure A.10: Rule catalogue at /rules: all seven rules with their phases and sources



Figure A.11: Health endpoint at /health, used by the cloud host to monitor the service

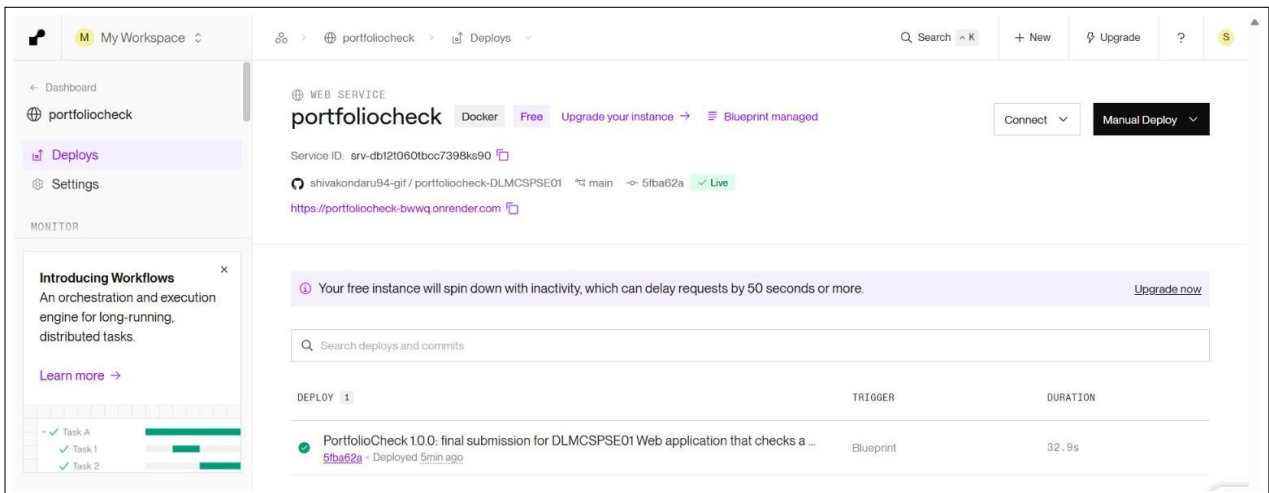


Figure A.12: The application deployed on Render

```
portfoliocheck - python -m pytest -v

$ python -m pytest -v
===== test session starts =====
collecting ... collected 73 items

tests/test_extraction.py::test_path_traversal_is_rejected PASSED [ 1%]
tests/test_extraction.py::test_absolute_path_is_rejected PASSED [ 2%]
tests/test_extraction.py::test_zip_bomb_is_rejected PASSED [ 4%]
tests/test_extraction.py::test_too_many_entries_is_rejected PASSED [ 5%]
tests/test_extraction.py::test_ordinary_archive_extracts PASSED [ 6%]
tests/test_repository.py::test_blob_hash_matches_git_for_known_content PASSED [ 8%]
tests/test_repository.py::test_blob_hash_of_empty_file_matches_git PASSED [ 9%]
tests/test_repository.py::test_blob_hash_agrees_with_git_hash_object PASSED [ 10%]
tests/test_repository.py::test_identical_archive_and_repository_pass PASSED [ 12%]
tests/test_repository.py::test_file_missing_from_archive_fails_and_is_named PASSED [ 13%]
tests/test_repository.py::test_extra_file_in_archive_fails PASSED [ 15%]
tests/test_repository.py::test_changed_file_content_fails PASSED [ 16%]
tests/test_repository.py::test_windows_line_endings_count_as_identical PASSED [ 17%]
tests/test_repository.py::test_github_download_zip_wrapper_folder_is_ignored PASSED [ 19%]
tests/test_repository.py::test_macos_metadata_and_git_folder_are_ignored PASSED [ 20%]
tests/test_repository.py::test_private_or_missing_repository_fails PASSED [ 21%]
tests/test_repository.py::test_unreachable_github_only_warns PASSED [ 23%]
tests/test_repository.py::test_truncated_tree_warns_even_when_files_match PASSED [ 24%]
tests/test_repository.py::test_missing_link_file_skips_comparison_with_warning PASSED [ 26%]
tests/test_repository.py::test_corrupt_code_archive_fails PASSED [ 27%]
tests/test_repository.py::test_nested_archive_with_path_traversal_is_rejected PASSED [ 28%]
tests/test_repository.py::test_repository_url_with_git_suffix_is_understood PASSED [ 30%]
tests/test_repository.py::test_comparison_not_run_for_phase_1 PASSED [ 31%]
tests/test_repository.py::test_client_builds_inventory_from_default_branch_tree PASSED [ 32%]
tests/test_repository.py::test_client_sends_token_when_configured PASSED [ 34%]
tests/test_repository.py::test_client_maps_404_to_not_found PASSED [ 35%]
tests/test_repository.py::test_client_maps_rate_limit_to_unavailable PASSED [ 36%]
tests/test_repository.py::test_client_maps_network_failure_to_unavailable PASSED [ 38%]
tests/test_rules.py::test_valid_submission_passes_every_rule PASSED [ 39%]
tests/test_rules.py::test_main_directory_name_must_match_pattern[MustermannMax-12345678-PSE] PASSED [ 41%]
tests/test_rules.py::test_main_directory_name_must_match_pattern[Mustermann-Max-12345678-PSE] PASSED [ 42%]
tests/test_rules.py::test_main_directory_name_must_match_pattern[Mustermann-Max-12345678] PASSED [ 43%]
tests/test_rules.py::test_main_directory_name_must_match_pattern[submission] PASSED [ 45%]
tests/test_rules.py::test_files_at_archive_root_are_rejected PASSED [ 46%]
tests/test_rules.py::test_unusual_matriculation_length_only_warns PASSED [ 47%]
tests/test_rules.py::test_unlaufs_in_surname_are_accepted PASSED [ 49%]
tests/test_rules.py::test_missing_subdirectory_fails PASSED [ 50%]
tests/test_rules.py::test_near_miss_folder_name_is_called_out PASSED [ 52%]
tests/test_rules.py::test_missing_code_archive_fails PASSED [ 53%]
tests/test_rules.py::test_wrongly_named_code_archive_fails PASSED [ 54%]
tests/test_rules.py::test_code_archive_prefix_mismatch_warns PASSED [ 56%]
tests/test_rules.py::test_code_archive_not_checked_in_phase_1 PASSED [ 57%]
tests/test_rules.py::test_missing_txt_file_fails PASSED [ 58%]
tests/test_rules.py::test_txt_without_github_url_fails PASSED [ 60%]
tests/test_rules.py::test_txt_without_application_url_warns PASSED [ 61%]
tests/test_rules.py::test_txt_with_invalid_encoding_does_not_crash PASSED [ 63%]
tests/test_rules.py::test_renamed_docx_is_detected PASSED [ 64%]
tests/test_rules.py::test_oversized_file_fails PASSED [ 65%]
tests/test_rules.py::test_failing_rule_does_not_stop_the_others PASSED [ 67%]
tests/test_rules.py::test_typical_submission_is_checked_within_three_seconds PASSED [ 68%]
tests/test_rules.py::test_every_finding_names_its_source_provision PASSED [ 69%]
tests/test_samples.py::test_sample_produces_documented_verdicts[M01] PASSED [ 71%]
tests/test_samples.py::test_sample_produces_documented_verdicts[M02] PASSED [ 72%]
tests/test_samples.py::test_sample_produces_documented_verdicts[M03] PASSED [ 73%]
tests/test_samples.py::test_sample_produces_documented_verdicts[M04] PASSED [ 75%]
tests/test_samples.py::test_sample_produces_documented_verdicts[M05] PASSED [ 76%]
tests/test_samples.py::test_sample_produces_documented_verdicts[M06] PASSED [ 78%]
tests/test_samples.py::test_sample_produces_documented_verdicts[M07] PASSED [ 79%]
tests/test_samples.py::test_compliant_phase1_sample_is_ready_to_submit PASSED [ 80%]
tests/test_samples.py::test_near_miss_sample_names_the_misspelt_folder PASSED [ 82%]
tests/test_samples.py::test_traversal_sample_is_rejected_before_extraction PASSED [ 83%]
tests/test_samples.py::test_non_zip_sample_is_rejected PASSED [ 84%]
tests/test_samples.py::test_sample_pdfs_are_structurally_valid PASSED [ 86%]
tests/test_web.py::test_landing_page_lists_the_rules PASSED [ 87%]
tests/test_web.py::test_valid_submission_reports_ready PASSED [ 89%]
tests/test_web.py::test_broken_submission_reports_failures PASSED [ 90%]
tests/test_web.py::test_non_zip_upload_is_rejected PASSED [ 91%]
tests/test_web.py::test_corrupt_zip_is_rejected PASSED [ 93%]
tests/test_web.py::test_skeleton_download_has_the_required_folders PASSED [ 94%]
tests/test_web.py::test_health PASSED [ 95%]
tests/test_web.py::test_rule_catalogue_lists_every_rule_with_its_source PASSED [ 97%]
tests/test_web.py::test_report_shows_source_provision PASSED [ 98%]
tests/test_web.py::test_unknown_phase_value_checks_all_rules PASSED [100%]

===== 73 passed in 8.98s =====
```

Figure A.13: Automated test run: all 73 tests pass